Programación Concurrente y Paralela

Práctica 5: *Software Transactional Memory*

2do Cuatrimestre 2026

Ejercicio 1. Modelar usando variables transaccionales un buffer de capacidad 1 que provee las operaciones `put` y `get`. Se utilizarán variables de tipo `TVar (Maybe t)` para representar buffers con valores de tipo `t`.

a) Definir las operaciones de manera tal que las mismas sean no bloqueantes, es decir, tienen el tipo:

```
get :: TVar (Maybe t) -> STM (Maybe t)
put :: TVar (Maybe t) -> t -> STM Bool
```

b) Dar un programa que crea un buffer con valor inicial 0 y luego ejecute concurrentemente una operación de escritura del valor 1 y una de lectura. El programa debe mostrar por pantalla el resultado que se obtiene de la lectura del buffer.

c) Dar una implementación de buffer donde las dos operaciones son bloqueantes, es decir, se debe esperar a que esté vacío para escribir y lleno para leer. Se espera que las operaciones tengan el siguiente tipo

```
get :: TVar (Maybe b) -> STM b
put :: TVar (Maybe t) -> t -> STM ()
```

d) Definir una una operación `read` que permite leer (sin tomar) el contenido del buffer . Si el buffer está vacío, la operación se bloquea hasta tanto se agregue algún valor.

Ejercicio 2. Implementar un semáforo en términos de STM. Mostrar un fragmento de código en el cual un cliente toma atómicamente dos permisos.

Ejercicio 3. Considerar el juego de las bolitas, donde existen dos tipos de participantes: los *generadores* y los *consumidores* de bolitas. Los generadores crean bolitas de a una a la vez y las colocan en una bolsa (suman un punto por cada bolita que logran colocar en la bolsa). Los consumidores toman pares de bolitas y suman un punto por cada par (pero no pueden tomar mas de un par por vez). No se conoce a priori la cantidad de participantes que están jugando, pero se debe garantizar en todo momento que la bolsa no contenga mas bolitas que la cantidad de generadores que se encuentran jugando.

Ejercicio 4. Dar una solución al problema de los fumadores. En el mismo hay tres procesos fumadores y un proceso proveedor. Cada uno de los procesos fumadores hará un cigarrillo y lo fumará. Para hacer un cigarrillo se requiere tabaco, papel y fósforos. Cada proceso fumador tiene uno de los tres elementos. Es decir, un proceso tiene tabaco, otro tiene papel y el tercero tiene fósforos. El proveedor tiene disponibilidad no acotada de cualquiera de los elementos. Periódicamente coloca dos elementos sobre la mesa, y el fumador que tiene el tercer elemento los toma y fuma su cigarrillo.

Ejercicio 5. Considerar el problema de la pizzería. En el bar X se fabrican dos tipos de pizzas: las chicas y las grandes. El maestro pizzero va colocando de a una las pizzas listas en una barra para que los clientes se sirvan y pasen luego por la caja. Lamentablemente todos los clientes tienen buen apetito y se comportan de la siguiente manera: prefieren siempre tomar un pizza grande, pero si no lo logran se conforman con dos pequeñas. Los clientes son mal educados y no esperan en una cola (todos compiten por las pizzas). Dar una solución utilizando STM en donde cada cliente y el maestro pizzero son modelados como threads.

Ejercicio 6. Implementar un tipo de datos Pila concurrente que permite la ejecución de las operaciones `push` y `pop`.

Ejercicio 7. Asumir la siguiente definición de tipos para representar una cola no acotada

```
data Cola a = Cola (TVar [a])
```

Se solicita:

a) Definir a las funciones para encolar y desencolar con los siguientes tipos:

```
encolar :: Cola a -> a -> STM ()
desencolar :: Cola a -> STM a
desencolarSi :: Cola a -> STM (Maybe a)
```

b) Definir una función que tome dos colas y atómicamente desencole un elemento de una y lo encole en la siguiente.

```
encolarDesencolar :: Cola a -> Cola a -> IO ()
```

c) Se desea implementar una cola con tres niveles de prioridad. Para ello se definen

```
data Prioridad = Alta | Media | Baja
data ColaPrioridad a = ColaPrioridad (Cola a) (Cola a) (Cola a)
```

donde los parámetros del constructor corresponden, en orden, a los elementos de alta, media y baja prioridad. Dar la definición de las funciones

```
encolarP :: ColaPrioridad a -> a -> Prioridad -> STM ()
desencolarP :: ColaPrioridad a -> STM a
```

La función `desencolarP` debe desencolar de la cola no vacía de mayor prioridad. Si todas están vacías, la operación debe bloquearse.

Ejercicio 8. Se desea implementar un lock de lectura/escritura utilizando STMs. Se cuenta con la siguiente definición del tipo de datos `RWLock` y la función que permite crear un `RWLock`.

```
data RWLock = RWLock (TVar Int) (TVar Bool)
newRWLock :: IO RWLock
newRWLock = do
  varLectores <- newTVarIO 0
  flagEscritor <- newTVarIO False
  return $ RWLock varLectores flagEscritor
```

donde el primer parámetro indica la cantidad de lectores y el segundo parámetro indica si hay un escritor.

a) Se pide definir las operaciones que permitan crear un lock sin para obtener y liberar el lock y además mostrar cómo implemetaría un thread que desee usar el lock en lectura.

b) Modificar la solución anterior de manera tal que sólo se permita una cantidad máxima de lectores concurrentes. Esta cantidad máxima está dada al momento de la creación del lock.

Ejercicio 9. Dar una solución al problema de los filósofos comensales.
Ayuda: Puede modelar a los tenedores como buffers (usando la resolución del ejercicio 1.b).